

SAM

后缀自动机

定义

S 的后缀自动机是一种能够识别所有 S 的子串的自动机类型的数据结构 (DFA)。

后缀自动机

定义

S 的后缀自动机是一种能够识别所有 S 的子串的自动机类型的数据结构 (DFA)。

自动机 = 状态集合 (state) + 状态转移 (trans) + 起始状态 + 终止状态 + 字符集。

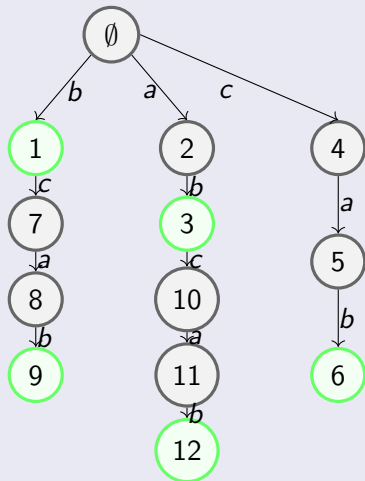
Naive 后缀自动机

Naive 版本

对于字符串 S ，将他的 $|S|$ 个后缀看作独立的串，构造 Trie 树，于是便可以识别所有后缀的前缀，即可以识别所有子串。

Naive 后缀自动机

$S = abcab$



Naive 版本的后缀自动机，状态数和转移数都为 $O(n^2)$ 。

最简状态后缀自动机

最简状态

Naive 版本的后缀自动机由于时空复杂度过高，不具有实用性。
那么最优的后缀自动机的状态数以及转移数是什么捏？

最简状态后缀自动机

最简状态

Naive 版本的后缀自动机由于时空复杂度过高，不具有实用性。

那么最优的后缀自动机的状态数以及转移数是什么捏？

答案：状态数 $O(2n)$ ，转移数 $O(3n)$ 。

那么这个最优的后缀自动机应该是什么样子捏？

最简状态后缀自动机

定义

$s(w)$ 表示子串 w 对应的后缀自动机上的状态。

$trans(s, ch)$ 表示当前状态是 s , 接收新字符 ch 之后到达的状态。

$Trans(s, str)$ 表示当前状态是 s , 接收新字符串 str 之后到达的状态。

$Suf[i]$ 表示从 i 位置开始的后缀, 即 $S[i, |S|]$ 。

最简状态后缀自动机

定义

$s(w)$ 表示子串 w 对应的后缀自动机上的状态。

$trans(s, ch)$ 表示当前状态是 s , 接收新字符 ch 之后到达的状态。

$Trans(s, str)$ 表示当前状态是 s , 接收新字符串 str 之后到达的状态。

$Suf[i]$ 表示从 i 位置开始的后缀, 即 $S[i, |S|]$ 。

一些简单的分析

假设某个子串 t 属于某个状态 s , 又设 t 的所有出现位置的右端点为 $r_1, r_2, r_3, \dots, r_m$ 。

最简状态后缀自动机

定义

$s(w)$ 表示子串 w 对应的后缀自动机上的状态。

$trans(s, ch)$ 表示当前状态是 s , 接收新字符 ch 之后到达的状态。

$Trans(s, str)$ 表示当前状态是 s , 接收新字符串 str 之后到达的状态。

$Suf[i]$ 表示从 i 位置开始的后缀, 即 $S[i, |S|]$ 。

一些简单的分析

假设某个子串 t 属于某个状态 s , 又设 t 的所有出现位置的右端点为 $r_1, r_2, r_3, \dots, r_m$ 。

则这个状态 s , 必然能接收后缀 $Suf[r_1 + 1], Suf[r_2 + 1], \dots, Suf[r_m + 1]$ 。

最简状态后缀自动机

定义

$s(w)$ 表示子串 w 对应的后缀自动机上的状态。

$trans(s, ch)$ 表示当前状态是 s , 接收新字符 ch 之后到达的状态。

$Trans(s, str)$ 表示当前状态是 s , 接收新字符串 str 之后到达的状态。

$Suf[i]$ 表示从 i 位置开始的后缀, 即 $S[i, |S|]$ 。

一些简单的分析

假设某个子串 t 属于某个状态 s , 又设 t 的所有出现位置的右端点为 $r_1, r_2, r_3, \dots, r_m$ 。

则这个状态 s , 必然能接收后缀 $Suf[r_1 + 1], Suf[r_2 + 1], \dots, Suf[r_m + 1]$ 。

同时, 如果另一个子串 t' 所在的状态 s' , 也只能接受

$Suf[r_1 + 1], Suf[r_2 + 1], \dots, Suf[r_m + 1]$, 即状态 s 和 s' 所能接受的输入相同, 从 DFA 的视角看, 他们必须是同一状态 $s' = s$ 。

最简状态后缀自动机

定义

$s(w)$ 表示子串 w 对应的后缀自动机上的状态。

$trans(s, ch)$ 表示当前状态是 s , 接收新字符 ch 之后到达的状态。

$Trans(s, str)$ 表示当前状态是 s , 接收新字符串 str 之后到达的状态。

$Suf[i]$ 表示从 i 位置开始的后缀, 即 $S[i, |S|]$ 。

一些简单的分析

假设某个子串 t 属于某个状态 s , 又设 t 的所有出现位置的右端点为 $r_1, r_2, r_3, \dots, r_m$ 。

则这个状态 s , 必然能接收后缀 $Suf[r_1 + 1], Suf[r_2 + 1], \dots, Suf[r_m + 1]$ 。

同时, 如果另一个子串 t' 所在的状态 s' , 也只能接受

$Suf[r_1 + 1], Suf[r_2 + 1], \dots, Suf[r_m + 1]$, 即状态 s 和 s' 所能接受的输入相同, 从 DFA 的视角看, 他们必须是同一状态 $s' = s$ 。

于是 t 和 t' 出现位置相同, 长度不同, 所以 t 是 t' 的后缀, 或者 t' 是 t 的后缀。

因此在后缀自动机中, 一个状态表示的是出现位置相同, 但长度不同的一系列串

最简状态后缀自动机

新定义

$Right(s) = \{r_1, r_2, \dots, r_m\}$, 表示 s 状态代表子串的出现位置右端点。

一些简单的推论

在最简状态后缀自动机中, 所有节点的 $Right$ 集合互不相同。

最简状态后缀自动机

新定义

$Right(s) = \{r_1, r_2, \dots, r_m\}$, 表示 s 状态代表子串的出现位置右端点。

一些简单的推论

在最简状态后缀自动机中, 所有节点的 $Right$ 集合互不相同。
每个节点代表的串之间形成后缀关系, 即所有串都是最长串的后缀。

最简状态后缀自动机

新定义

$Right(s) = \{r_1, r_2, \dots, r_m\}$, 表示 s 状态代表子串的出现位置右端点。

一些简单的推论

在最简状态后缀自动机中, 所有节点的 $Right$ 集合互不相同。
每个节点代表的串之间形成后缀关系, 即所有串都是最长串的后缀。
每个节点代表的串的长度是连续区间, 记为 $[MinL(s), MaxL(s)]$ 。

最简状态后缀自动机

新定义

$Right(s) = \{r_1, r_2, \dots, r_m\}$, 表示 s 状态代表子串的出现位置右端点。

一些简单的推论

在最简状态后缀自动机中, 所有节点的 $Right$ 集合互不相同。
每个节点代表的串之间形成后缀关系, 即所有串都是最长串的后缀。
每个节点代表的串的长度是连续区间, 记为 $[MinL(s), MaxL(s)]$ 。
考虑同在 $Right(s)$ 位置的长度为 $MinL(s) - 1$ 的子串, 设其所在的状态为 s' , 则必有 $Right(s) \subsetneq Right(s')$

最简状态后缀自动机

Right 的性质

设两个状态 s 和 s' , 其 *Right* 集合分别为 $R(s)$ 和 $R(s')$ 。

假设 $R(s) \cap R(s') \neq \emptyset$, 并且 $r \in R(s) \cap R(s')$ 。

由于任何两个状态代表的串互不相同, 不失一般性, 可以认为 s 代表的串都比 s' 代表的串长。

因此必然有 $R(s) \subsetneq R(s')$ 。

最简状态后缀自动机

Right 的性质

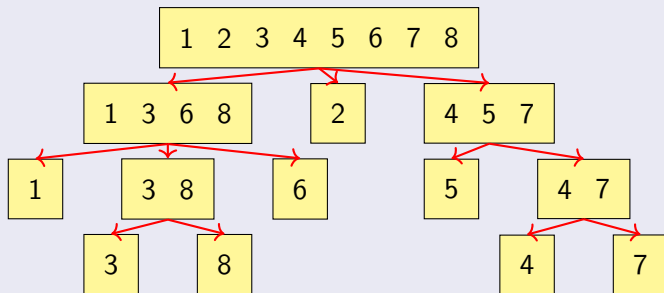
设两个状态 s 和 s' ，其 $Right$ 集合分别为 $R(s)$ 和 $R(s')$ 。

假设 $R(s) \cap R(s') \neq \emptyset$ ，并且 $r \in R(s) \cap R(s')$ 。

由于任何两个状态代表的串互不相同，不失一般性，可以认为 s 代表的串都比 s' 代表的串长。

因此必然有 $R(s) \subsetneq R(s')$ 。

结论：两个不同状态的 $Right$ 集合只存在两种关系：不相交，或者真包含。即形成一种树形结构。



最简状态后缀自动机

新新定义

$SC(\text{suffix-chain})$ 树（后缀链接树）即为 $Right$ 集合包含关系形成的树。（在一些不是很严谨的文章中，也称为 $Parent$ 树， $link$ 树等。） $f(s)$ 表示状态 s 对应的 $Right(s)$ 在 SC 树上的父节点。

最简状态后缀自动机

新新定义

$SC(\text{suffix-chain})$ 树（后缀链接树）即为 $Right$ 集合包含关系形成的树。（在一些不是很严谨的文章中，也称为 $Parent$ 树， $link$ 树等。） $f(s)$ 表示状态 s 对应的 $Right(s)$ 在 SC 树上的父节点。

后缀链接 (SC) 树的性质

每个前缀所在的状态两两不同。

最简状态后缀自动机

新新定义

$SC(\text{suffix-chain})$ 树（后缀链接树）即为 $Right$ 集合包含关系形成的树。（在一些不是很严谨的文章中，也称为 $Parent$ 树， $link$ 树等。） $f(s)$ 表示状态 s 对应的 $Right(s)$ 在 SC 树上的父节点。

后缀链接 (SC) 树的性质

每个前缀所在的状态两两不同。

共有 $|S|$ 个叶子节点，分别对应于每个前缀 $S[1, i]$ 所在的状态。

最简状态后缀自动机

新新定义

$SC(\text{suffix-chain})$ 树（后缀链接树）即为 $Right$ 集合包含关系形成的树。（在一些不是很严谨的文章中，也称为 $Parent$ 树， $link$ 树等。）
 $f(s)$ 表示状态 s 对应的 $Right(s)$ 在 SC 树上的父节点。

后缀链接 (SC) 树的性质

每个前缀所在的状态两两不同。

共有 $|S|$ 个叶子节点，分别对应于每个前缀 $S[1, i]$ 所在的状态。

后缀链接树至多有 $2|S| - 1$ 个节点，即至多有这么多不同的 $Right$ 集合，即后缀自动机节点个数为 $O(2n)$ 。

最简状态后缀自动机

新新定义

$SC(\text{suffix-chain})$ 树 (后缀链接树) 即为 $Right$ 集合包含关系形成的树。(在一些不是很严谨的文章中, 也称为 $Parent$ 树, $link$ 树等。) $f(s)$ 表示状态 s 对应的 $Right(s)$ 在 SC 树上的父节点。

后缀链接 (SC) 树的性质

每个前缀所在的状态两两不同。

共有 $|S|$ 个叶子节点, 分别对应于每个前缀 $S[1, i]$ 所在的状态。

后缀链接树至多有 $2|S| - 1$ 个节点, 即至多有这么多不同的 $Right$ 集合, 即后缀自动机节点个数为 $O(2n)$ 。

任意串 w 的后缀全部位于 $s(w)$ 的后缀链接路径上。

最简状态后缀自动机

新新定义

$SC(\text{suffix-chain})$ 树（后缀链接树）即为 $Right$ 集合包含关系形成的树。（在一些不是很严谨的文章中，也称为 $Parent$ 树， $link$ 树等。） $f(s)$ 表示状态 s 对应的 $Right(s)$ 在 SC 树上的父节点。

后缀链接 (SC) 树的性质

每个前缀所在的状态两两不同。

共有 $|S|$ 个叶子节点，分别对应于每个前缀 $S[1, i]$ 所在的状态。

后缀链接树至多有 $2|S| - 1$ 个节点，即至多有这么多不同的 $Right$ 集合，即后缀自动机节点个数为 $O(2n)$ 。

任意串 w 的后缀全部位于 $s(w)$ 的后缀链接路径上。

若某个状态 s 拥有 ch 的转移边，那么 $f[s]$ 也一定有 ch 的转移边。（但不一定转移到同一个状态）

最简状态后缀自动机

新新定义

$SC(\text{suffix-chain})$ 树（后缀链接树）即为 $Right$ 集合包含关系形成的树。（在一些不是很严谨的文章中，也称为 $Parent$ 树， $link$ 树等。） $f(s)$ 表示状态 s 对应的 $Right(s)$ 在 SC 树上的父节点。

后缀链接 (SC) 树的性质

每个前缀所在的状态两两不同。

共有 $|S|$ 个叶子节点，分别对应于每个前缀 $S[1, i]$ 所在的状态。

后缀链接树至多有 $2|S| - 1$ 个节点，即至多有这么多不同的 $Right$ 集合，即后缀自动机节点个数为 $O(2n)$ 。

任意串 w 的后缀全部位于 $s(w)$ 的后缀链接路径上。

若某个状态 s 拥有 ch 的转移边，那么 $f[s]$ 也一定有 ch 的转移边。（但不一定转移到同一个状态）

每个状态 s 的 $Right(s)$ 等价于他在后缀链接树子树的叶子节点集合。

最简状态后缀自动机

一些显然的观察

对于任意状态 s , $MaxL[f(s)] = MinL[s] - 1$ 。

因此每个状态只需要额外记录 $L[s] = MaxL[s]$ 以及 $f(s)$, 于是状态 s 能表示长度为 $(L[f(s)], L[s]]$ 的串。

所有终止状态都能代表至少一个后缀。

$Right$ 集合概念等价于子树概念, 因此不需要额外维护。

最简状态后缀自动机

一些显然的观察

对于任意状态 s , $MaxL[f(s)] = MinL[s] - 1$ 。

因此每个状态只需要额外记录 $L[s] = MaxL[s]$ 以及 $f(s)$, 于是状态 s 能表示长度为 $(L[f(s)], L[s]]$ 的串。

所有终止状态都能代表至少一个后缀。

Right 集合概念等价于子树概念, 因此不需要额外维护。

现在搞清楚了最简状态后缀自动机中只会有不超过 $O(2|S|)$ 个状态, 以及每个状态代表了哪些串。

那么状态转移个数, 即 $trans(s, ch)$ 总数是多少捏?

最简状态后缀自动机

状态转移

由于所有状态必须是 $root$ 可达的，因此存在以 $root$ 为根的生成树，由于点数只有 $2|S| - 1$ ，因此树边最多 $2|S| - 2$ ，接下来考虑非树边有多少条。

最简状态后缀自动机

状态转移

由于所有状态必须是 $root$ 可达的，因此存在以 $root$ 为根的生成树，由于点数只有 $2|S| - 1$ ，因此树边最多 $2|S| - 2$ ，接下来考虑非树边有多少条。

考虑如下过程：对于每条非树边 $trans(u, ch) = v$ ，我们可以轻松得到包含他的一个后缀，放法如下：

1. 在生成树上从 $root$ 走到 u 。
2. 走 $trans(u, ch) = v$ 的非树边。
3. 在后缀自动机（而非在生成树上）上任意地继续游走，直到走到终止节点，即任意一个后缀所在的状态。

最简状态后缀自动机

状态转移

由于所有状态必须是 $root$ 可达的，因此存在以 $root$ 为根的生成树，由于点数只有 $2|S| - 1$ ，因此树边最多 $2|S| - 2$ ，接下来考虑非树边有多少条。

考虑如下过程：对于每条非树边 $trans(u, ch) = v$ ，我们可以轻松得到包含他的一个后缀，放法如下：

1. 在生成树上从 $root$ 走到 u 。
2. 走 $trans(u, ch) = v$ 的非树边。
3. 在后缀自动机（而非在生成树上）上任意地继续游走，直到走到终止节点，即任意一个后缀所在的状态。

于是我们可以将每一条非树边都映射到一个后缀上，如果我们能证明每个后缀只会被一条非树边映射到，那么非树边的个数就不超过后缀的数量了。证明如下：

最简状态后缀自动机

状态转移

由于所有状态必须是 $root$ 可达的，因此存在以 $root$ 为根的生成树，由于点数只有 $2|S| - 1$ ，因此树边最多 $2|S| - 2$ ，接下来考虑非树边有多少条。

考虑如下过程：对于每条非树边 $trans(u, ch) = v$ ，我们可以轻松得到包含他的一个后缀，放法如下：

1. 在生成树上从 $root$ 走到 u 。
2. 走 $trans(u, ch) = v$ 的非树边。
3. 在**后缀自动机**（而非在生成树上）上任意地继续游走，直到走到终止节点，即任意一个后缀所在的状态。

于是我们可以将每一条非树边都映射到一个后缀上，如果我们能证明每个后缀只会被一条非树边映射到，那么非树边的个数就不超过后缀的数量了。证明如下：

根据 DFA 的性质，走到每个后缀的路径在自动机上是唯一确定的，所以上面建立的（非树边 $\rightarrow$ 后缀）的映射满足性质：该非树边是该后缀使用的**第一条**非树边。对一个后缀而言，第一条只会有一条，证毕。。。因此状态转移个数是 $O(3|S|)$ 。

小结

最简状态后缀自动机

1. 状态个数以及转移边个数都是线性。
2. 每个状态 s 能够表示以任意 $R(s)$ 中的位置作为右端点的，长度范围在 $(L[f(s)], L[s]]$ 的子串。每个状态可以看作是一个 *Right* 意义上的等价类。
3. 在构造过程中，需要维护每个状态的 $f(s)$ ， $L[s]$ ，以及 $trans(s, ch)$ 。（*Right* 集合不需要真的去维护，转化为子树查询即可。）

小结

最简状态后缀自动机

1. 状态个数以及转移边个数都是线性。
2. 每个状态 s 能够表示以任意 $R(s)$ 中的位置作为右端点的，长度范围在 $(L[f(s)], L[s]]$ 的子串。每个状态可以看作是一个 *Right* 意义上的等价类。
3. 在构造过程中，需要维护每个状态的 $f(s)$, $L[s]$, 以及 $trans(s, ch)$ 。(*Right* 集合不需要真的去维护，转化为子树查询即可。) 接下来的主要问题就是：到底怎么构造这么一个玩意捏？

构造后缀自动机

在线增量构造

首先创建第一个节点 (标号为 1) 表示 *root* (空子串 $\emptyset$), $L[1] = 0$, $f(1) = 0(\text{null})$, 且 $R(1)$ 为全集。

接下来逐个读入 S 的每个字符 $S[i]$, 然后对已有的 $S[1, i-1]$ 的自动机进行一系列调整, 得到 $S[1, i]$ 的自动机。

构造后缀自动机

在线增量构造

首先创建第一个节点 (标号为 1) 表示 *root* (空子串 $\emptyset$), $L[1] = 0$, $f(1) = 0(\text{null})$, 且 $R(1)$ 为全集。

接下来逐个读入 S 的每个字符 $S[i]$, 然后对已有的 $S[1, i-1]$ 的自动机进行一系列调整, 得到 $S[1, i]$ 的自动机。

目标

思考 $S[1, i]$ 的自动机与 $S[1, i-1]$ 的自动机有什么差别?

构造后缀自动机

在线增量构造

首先创建第一个节点 (标号为 1) 表示 $root$ (空子串 $\emptyset$), $L[1] = 0$, $f(1) = 0(null)$, 且 $R(1)$ 为全集。

接下来逐个读入 S 的每个字符 $S[i]$, 然后对已有的 $S[1, i-1]$ 的自动机进行一系列调整, 得到 $S[1, i]$ 的自动机。

目标

思考 $S[1, i]$ 的自动机与 $S[1, i-1]$ 的自动机有什么差别?

答案是新增了 i 个子串, 即 $S[1, i]$ 的所有后缀, 我们要确保这些新的子串都能找到对应的状态, 并且保证这些状态的转移边是完备的, 即每个新子串都可以从根沿着 $trans$ 走到, 同时要保证每个状态的 f, L 值正确。

构造后缀自动机

为 $S[1, i]$ 创建新状态

我们从长到短的来考虑新增的 i 个子串，即 $S[1, i]$ 的所有后缀。
首先可以确定， $S[1, i]$ 一定是自动机里不存在的串，因为太长了，所以必须新建一个新状态 np 。

构造后缀自动机

为 $S[1, i]$ 创建新状态

我们从长到短的来考虑新增的 i 个子串，即 $S[1, i]$ 的所有后缀。
首先可以确定， $S[1, i]$ 一定是自动机里不存在的串，因为太长了，所以必须新建一个新状态 np 。
这个新状态目前的 **trans 为空**， **$L[np] = i$** 。

构造后缀自动机

为 $S[1, i]$ 创建新状态

我们从长到短的来考虑新增的 i 个子串，即 $S[1, i]$ 的所有后缀。
首先可以确定， $S[1, i]$ 一定是自动机里不存在的串，因为太长了，所以必须新建一个新状态 np 。

这个新状态目前的 **trans 为空**， **$L[np] = i$** 。

接下来需要确定新状态的 $f(np)$ 指向，以及有哪些已存在状态需要新增转移边 $trans(s, S[i]) = np$ 。

构造后缀自动机

寻找 $f(np)$

由于 $Right(np) = \{i\}$, 根据 *Suffix-Chain* 树的定义可知, 我们要寻找 $S[1, i]$ 最长的后缀 $S[i - x + 1, i]$, 其出现位置不止有 i 。
($|Right(s(S[1 - x + 1, i]))| > 1$).

构造后缀自动机

寻找 $f(np)$

由于 $Right(np) = \{i\}$, 根据 *Suffix-Chain* 树的定义可知, 我们要寻找 $S[1, i]$ 最长的后缀 $S[i - x + 1, i]$, 其出现位置不止有 i 。
($|Right(s(S[1 - x + 1, i]))| > 1$).

即寻找最长的 $S[i - x + 1, i]$, 要求在 $S[1, i - 1]$ 中出现过。

构造后缀自动机

寻找 $f(np)$

由于 $Right(np) = \{i\}$, 根据 *Suffix-Chain* 树的定义可知, 我们要寻找 $S[1, i]$ 最长的后缀 $S[i - x + 1, i]$, 其出现位置不止有 i .
($|Right(s(S[1 - x + 1, i]))| > 1$).

即寻找最长的 $S[i - x + 1, i]$, 要求在 $S[1, i - 1]$ 中出现过。

由于 $S[i - x + 1, i] = S[i - x + 1, i - 1] + S[i]$, 因此我们就是要寻找 $S[1, i - 1]$ 的最长后缀, 且要求其所在的状态拥有 $S[i]$ 的转移边。

构造后缀自动机

寻找 $f(np)$

由于 $Right(np) = \{i\}$, 根据 *Suffix-Chain* 树的定义可知, 我们要寻找 $S[1, i]$ 最长的后缀 $S[i - x + 1, i]$, 其出现位置不止有 i .
($|Right(s(S[1 - x + 1, i]))| > 1$).

即寻找最长的 $S[i - x + 1, i]$, 要求在 $S[1, i - 1]$ 中出现过。

由于 $S[i - x + 1, i] = S[i - x + 1, i - 1] + S[i]$, 因此我们就是要寻找 $S[1, i - 1]$ 的最长后缀, 且要求其所在的状态拥有 $S[i]$ 的转移边。
如此一来, np 节点表示的串长度范围将是 $[x + 1, i]$ 。

构造后缀自动机

寻找 $f(np)$

由于 $Right(np) = \{i\}$, 根据 *Suffix-Chain* 树的定义可知, 我们要寻找 $S[1, i]$ 最长的后缀 $S[i - x + 1, i]$, 其出现位置不止有 i 。
($|Right(s(S[1 - x + 1, i]))| > 1$).

即寻找最长的 $S[i - x + 1, i]$, 要求在 $S[1, i - 1]$ 中出现过。

由于 $S[i - x + 1, i] = S[i - x + 1, i - 1] + S[i]$, 因此我们就是要寻找 $S[1, i - 1]$ 的最长后缀, 且要求其所在的状态拥有 $S[i]$ 的转移边。

如此一来, np 节点表示的串长度范围将是 $[x + 1, i]$ 。

这一过程要求我们枚举所有 $S[1, i - 1]$ 的后缀, 设 $S[1, i - 1]$ 所在的状态是 p , 这一步可以在 *Suffix-Chain* 树上爬 p 到 $root$ 的链来实现。

构造后缀自动机

寻找 $f(np)$

显然，我们这样做会有两种结果：

构造后缀自动机

寻找 $f(np)$

显然，我们这样做会有两种结果：

1. p 到 $root$ 的树链上所有状态都没有 $S[i]$ 的转移边，即所有 $S[1, i]$ 的后缀在 $S[1, i-1]$ 中都没有出现过，即他们都只在 i 位置出现，因此他们都可以被状态 np 表示，此时 $f(np) = root$ 。

构造后缀自动机

寻找 $f(np)$

显然，我们这样做会有两种结果：

1. p 到 $root$ 的树链上所有状态都没有 $S[i]$ 的转移边，即所有 $S[1, i]$ 的后缀在 $S[1, i-1]$ 中都没有出现过，即他们都只在 i 位置出现，因此他们都可以被状态 np 表示，此时 $f(np) = root$ 。
2. 从 p 向 $root$ 遍历的过程中，找到了第一个带有 $S[i]$ 转移边的状态 p' ，设 $trans(p', S[i]) = q$ 。这意味着我们找到了 np 状态无法表示的，最长的 $S[1, i]$ 后缀，其长度为 $L[p'] + 1$ 。那么此时 $f(np)$ 应该是什么呢？是 q 吗？

构造后缀自动机

寻找 $f(np)$

显然，我们这样做会有两种结果：

1. p 到 $root$ 的树链上所有状态都没有 $S[i]$ 的转移边，即所有 $S[1, i]$ 的后缀在 $S[1, i-1]$ 中都没有出现过，即他们都只在 i 位置出现，因此他们都可以被状态 np 表示，此时 $f(np) = root$ 。
2. 从 p 向 $root$ 遍历的过程中，找到了第一个带有 $S[i]$ 转移边的状态 p' ，设 $trans(p', S[i]) = q$ 。这意味着我们找到了 np 状态无法表示的，最长的 $S[1, i]$ 后缀，其长度为 $L[p'] + 1$ 。那么此时 $f(np)$ 应该是什么呢？是 q 吗？

暂且按下不表。

构造后缀自动机

补全 $trans(s, S[i])$

由于所有新串都是 $S[1, i]$ 的后缀, 即 $S[j, i] = S[j, i-1] + S[i]$, 因此我们需要确保 $S[1, i-1]$ 的每个后缀所在的状态 s , 都要有 $trans(s, S[i])$ 转移边。

构造后缀自动机

补全 $trans(s, S[i])$

由于所有新串都是 $S[1, i]$ 的后缀, 即 $S[j, i] = S[j, i-1] + S[i]$, 因此我们需要确保 $S[1, i-1]$ 的每个后缀所在的状态 s , 都要有 $trans(s, S[i])$ 转移边。从长到短遍历 $S[1, i-1]$ 的后缀等价于在 *Suffix-Chain* 树上爬 p 到 $root$ 的链。

构造后缀自动机

补全 $trans(s, S[i])$

由于所有新串都是 $S[1, i]$ 的后缀，即 $S[j, i] = S[j, i-1] + S[i]$ ，因此我们需要确保 $S[1, i-1]$ 的每个后缀所在的状态 s ，都要有 $trans(s, S[i])$ 转移边。从长到短遍历 $S[1, i-1]$ 的后缀等价于在 *Suffix-Chain* 树上爬 p 到 $root$ 的链。

显然，我们这样做会有两种结果：

1. p 到 $root$ 的树链上所有状态都没有 $S[i]$ 的转移边，我们要做的只是给他们补上 $trans(s, S[i]) = np$ 即可。

构造后缀自动机

补全 $trans(s, S[i])$

由于所有新串都是 $S[1, i]$ 的后缀，即 $S[j, i] = S[j, i-1] + S[i]$ ，因此我们需要确保 $S[1, i-1]$ 的每个后缀所在的状态 s ，都要有 $trans(s, S[i])$ 转移边。从长到短遍历 $S[1, i-1]$ 的后缀等价于在 *Suffix-Chain* 树上爬 p 到 $root$ 的链。

显然，我们这样做会有两种结果：

1. p 到 $root$ 的树链上所有状态都没有 $S[i]$ 的转移边，我们要做的只是给他们补上 $trans(s, S[i]) = np$ 即可。
2. 从 p 向 $root$ 遍历的过程中，找到了第一个带有 $S[i]$ 转移边的状态 p' 。我们本来要给 p' 添加指向 np 的 $S[i]$ 转移边，但是已经有 $trans(p', S[i]) = q$ 了，该怎么办呢，怎么办呢？

构造后缀自动机

补全 $trans(s, S[i])$

由于所有新串都是 $S[1, i]$ 的后缀，即 $S[j, i] = S[j, i-1] + S[i]$ ，因此我们需要确保 $S[1, i-1]$ 的每个后缀所在的状态 s ，都要有 $trans(s, S[i])$ 转移边。从长到短遍历 $S[1, i-1]$ 的后缀等价于在 *Suffix-Chain* 树上爬 p 到 $root$ 的链。

显然，我们这样做会有两种结果：

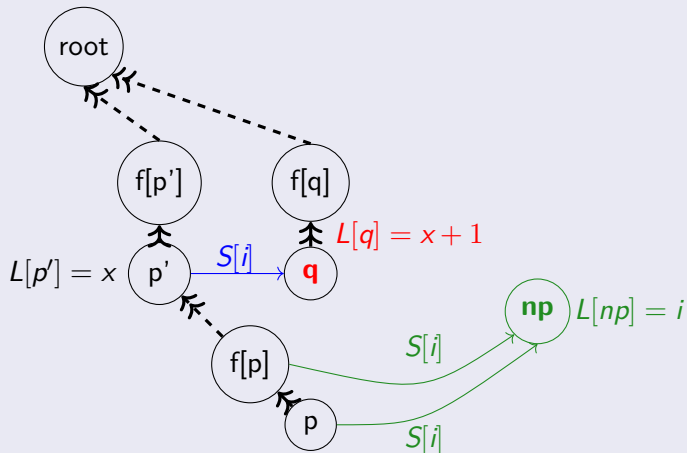
1. p 到 $root$ 的树链上所有状态都没有 $S[i]$ 的转移边，我们要做的只是给他们补上 $trans(s, S[i]) = np$ 即可。
2. 从 p 向 $root$ 遍历的过程中，找到了第一个带有 $S[i]$ 转移边的状态 p' 。我们本来要给 p' 添加指向 np 的 $S[i]$ 转移边，但是已经有 $trans(p', S[i]) = q$ 了，该怎么办呢，怎么办呢？

两个任务遇到了相同的问题，伏笔回收。

构造后缀自动机

Case 1. $L[q] = L[p'] + 1$

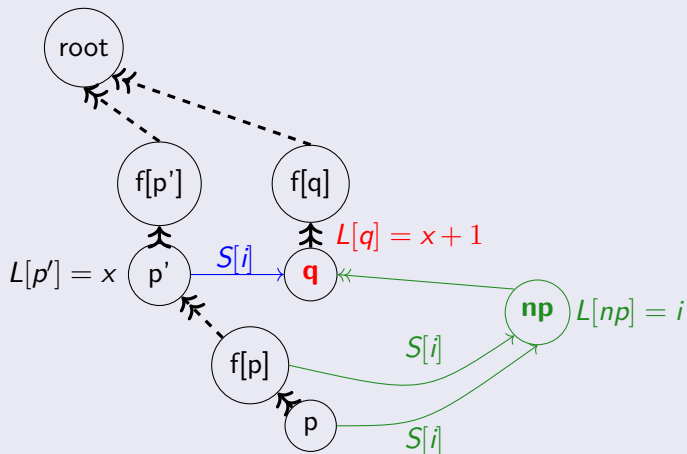
由前面的分析可知，新状态 np 不能表示的最大串长度就是 $x+1$ 。此时只需要令 $f(np) = q$ **即可**。



构造后缀自动机

Case 1. $L[q] = L[p'] + 1$

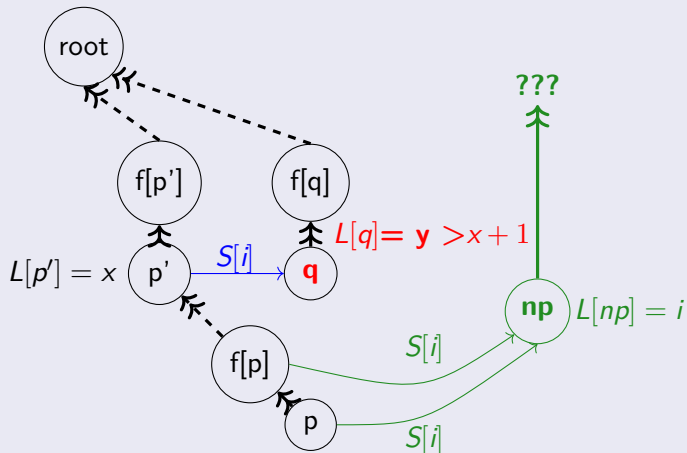
由前面的分析可知，新状态 np 不能表示的最大串长度就是 $x+1$ 。此时只需要令 $f(np) = q$ 即可。想想为什么？



构造后缀自动机

Case 2. $L[q] \neq L[p'] + 1$

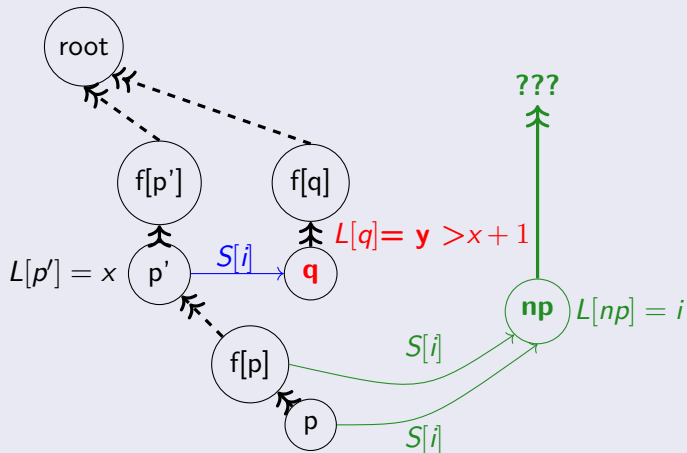
即 $L[q] > L[p'] + 1$ 。由前面的分析可知，新状态的父状态 $f(np)$ 应满足 $L[f(np)] = x + 1$ 。此时不能令 $f(np)$ 直接等于 q ，那么怎么办捏？



构造后缀自动机

Case 2. $L[q] \neq L[p'] + 1$

直觉告诉我们，要将 q 状态分裂出一个新状态 np ，表示原来 q 状态中较短 ($\leq x+1$) 的一些串。



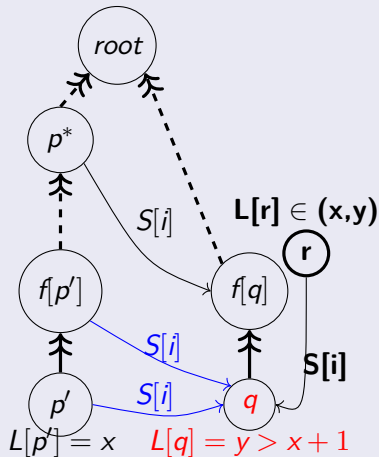
构造后缀自动机

分裂 q 状态

除了将状态 q 的整个 $trans$ 转移复制给分裂状态 nq 外，还需要调整一系列原本 $trans(s, S[i]) = q$ 的转移边变为 $trans(s, S[i]) = nq$ 。

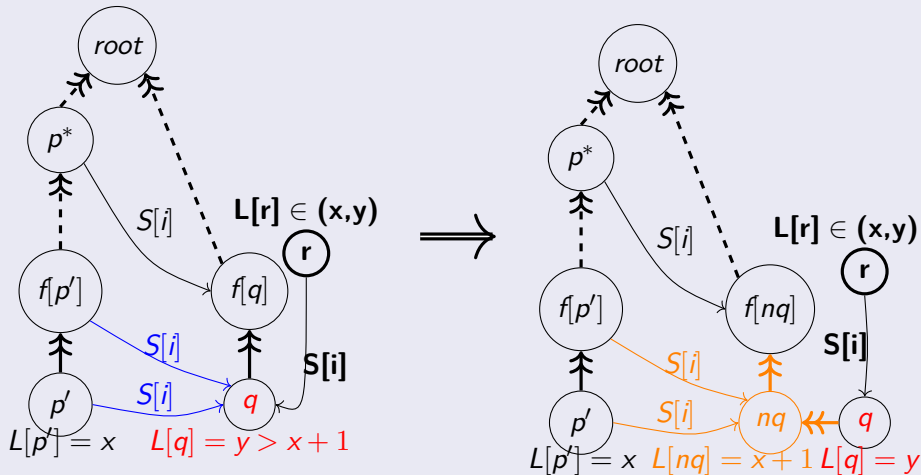
构造后缀自动机

分裂 q 状态



构造后缀自动机

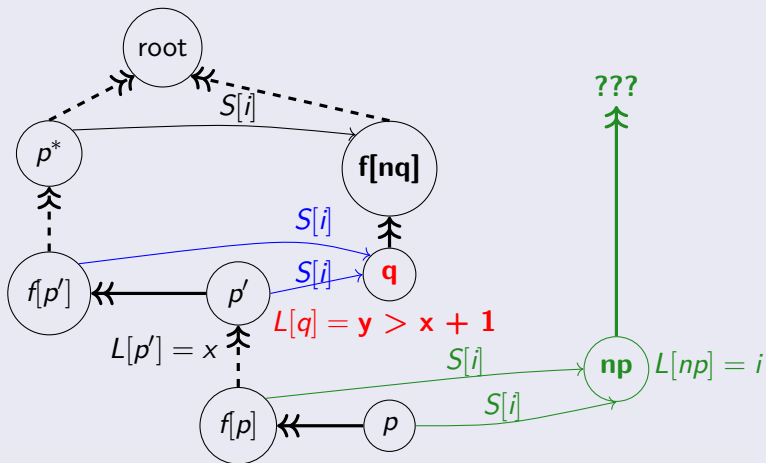
分裂 q 状态



构造后缀自动机

Case 2. $L[q] \neq L[p'] + 1$

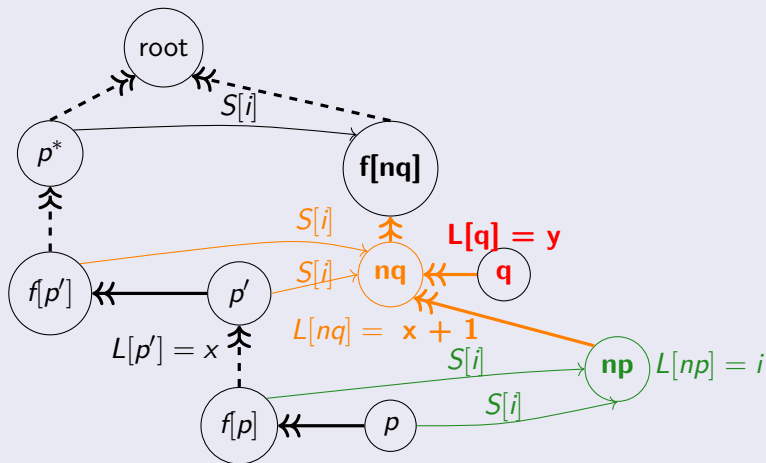
将 q 节点分裂之后, $f(np) = nq$ 即可。



构造后缀自动机

Case 2. $L[q] \neq L[p'] + 1$

将 q 节点分裂之后, $f(np) = nq$ 即可。



复杂度分析

空间复杂度

$O(2n)$ 个状态

$trans[s][ch]$ 数组存状态转移, $O(\Sigma n)$ 。

$f[s]$ 存 *Suffix-Chain* 树, $O(n)$ 。

$L[s]$ 记录每个状态表示的最大串长度, $O(n)$ 。

复杂度分析

空间复杂度

$O(2n)$ 个状态

$trans[s][ch]$ 数组存状态转移, $O(\Sigma n)$ 。

$f[s]$ 存 *Suffix-Chain* 树, $O(n)$ 。

$L[s]$ 记录每个状态表示的最大串长度, $O(n)$ 。

时间复杂度

令 $depth[s]$ 表示状态 s 在 *Suffix-Chain* 树上的深度, 则可以证明:
 $depth[np] \leq depth[p] - K + 2$, 其中 K 是暴力加/改转移边的次数。

复杂度分析

空间复杂度

$O(2n)$ 个状态

$trans[s][ch]$ 数组存状态转移, $O(\Sigma n)$ 。

$f[s]$ 存 *Suffix-Chain* 树, $O(n)$ 。

$L[s]$ 记录每个状态表示的最大串长度, $O(n)$ 。

时间复杂度

令 $depth[s]$ 表示状态 s 在 *Suffix-Chain* 树上的深度, 则可以证明:

$depth[np] \leq depth[p] - K + 2$, 其中 K 是暴力加/改转移边的次数。

因此定义势能为 $depth[np] - depth[p]$, 可知势能总量为 $2|S|$, 于是暴力

while 的部分复杂度为线性, 其他部分复杂度很容易看出也是线性。

强烈建议阅读[原论文](#)

后缀自动机

名词对照

- 后缀自动机, *SAM(Suffix Automaton)*, **DAWG(单词的有向无环图)**
- Parent 树, link 树, ***SC(Suffix Chain)*** 后缀链接
- 状态, 等价类
- *Right* 集合, end-positions, ***end-set***

其中**加粗**的为原论文中的名词。

例题 1-求最小循环串

题意

给出一个串 S ，每次操作可以将 S 的最前面一个字符移动到 S 的末尾，可以执行任意次操作，求得到的字典序最小串是什么。

例题 1-求最小循环串

题意

给出一个串 S ，每次操作可以将 S 的最前面一个字符移动到 S 的末尾，可以执行任意次操作，求得到的字典序最小串是什么。

将

S 复制一份拼接在后面得到 $T = S + S$ ，对 T 构造 SAM，然后在上面走 $|S|$ 步，每步都走最小的转移边即可。

例题 1-求最小循环串

题意

给出一个串 S ，每次操作可以将 S 的最前面一个字符移动到 S 的末尾，可以执行任意次操作，求得到的字典序最小串是什么。

将

S 复制一份拼接在后面得到 $T = S + S$ ，对 T 构造 SAM，然后在上面走 $|S|$ 步，每步都走最小的转移边即可。
想想为什么不会提前遇到中止节点。

例题 2-SPOJ NSUBSTR

题意

给出一个字符串 S , $|S| \leq 250,000$ 。

令 $G[x]$ 表示 S 的所有长度为 x 的子串中, 出现次数的最大值。

求 $G[1], G[2], \dots, G[|S|]$ 。

例题 2-SPOJ NSUBSTR

题意

给出一个字符串 S , $|S| \leq 250,000$ 。

令 $G[x]$ 表示 S 的所有长度为 x 的子串中, 出现次数的最大值。

求 $G[1], G[2], \dots, G[|S|]$ 。

题解

构建 S 的 SAM, 对于每个 SAM 状态 s , 他能表示的子串长度范围是 $(L[f(s)], L[s]]$, 这些子串的出现次数等价于 $|Right(s)|$ 。那么问题变成用 $|Right(s)|$ 去区间更新 $(L[f(s)], L[s]]$ 的最大值。

例题 2-SPOJ NSUBSTR

题意

给出一个字符串 S , $|S| \leq 250,000$ 。

令 $G[x]$ 表示 S 的所有长度为 x 的子串中, 出现次数的最大值。

求 $G[1], G[2], \dots, G[|S|]$ 。

题解

构建 S 的 SAM, 对于每个 SAM 状态 s , 他能表示的子串长度范围是 $(L[f(s)], L[s]]$, 这些子串的出现次数等价于 $|Right(s)|$ 。那么问题变成用 $|Right(s)|$ 去区间更新 $(L[f(s)], L[s]]$ 的最大值。

无脑线段树? 喜提 TLE。怎么优化呢

例题 2-SPOJ NSUBSTR

题意

给出一个字符串 S , $|S| \leq 250,000$ 。

令 $G[x]$ 表示 S 的所有长度为 x 的子串中, 出现次数的最大值。

求 $G[1], G[2], \dots, G[|S|]$ 。

题解

构建 S 的 SAM, 对于每个 SAM 状态 s , 他能表示的子串长度范围是 $(L[f(s)], L[s]]$, 这些子串的出现次数等价于 $|Right(s)|$ 。那么问题变成用 $|Right(s)|$ 去区间更新 $(L[f(s)], L[s]]$ 的最大值。

无脑线段树? 喜提 TLE。怎么优化呢

显然 $G[i]$ 是单调不减的, 我们只用 $|Right(s)|$ 去更新 $G[L[s]]$, 然后使用推标记的放法, 从后向前令 $G[i] = \max(G[i], G[i+1])$ 即可。

求 $|Right[s]|$

大常数做法

前面说过，不存在两个前缀位于同一个状态中，而每个前缀 $S[1, i]$ 的 $Right$ 集合必然包括了 i ，因此我们在构建 SAM 的时候，顺便维护 $count[np] = |Right[np]| = 1$ 。然后运行一遍 dfs ，求子树之和即可。

求 $|Right[s]|$

大常数做法

前面说过，不存在两个前缀位于同一个状态中，而每个前缀 $S[1, i]$ 的 $Right$ 集合必然包括了 i ，因此我们在构建 SAM 的时候，顺便维护 $count[np] = |Right[np]| = 1$ 。然后运行一遍 dfs ，求子树之和即可。

小常数做法

dfs 本质为树 DP，树 DP 本质是按照拓扑序来进行 DP，*Suffix-Chain* 树的拓扑序等价于按照所有状态的 $L[s]$ 从大到小排序 $\rightarrow$ 基数排序。

例题 3-快速定位 $s(S[l, r])$

题意

给出一个字符串 S , $|S| \leq 200,000$, 给出 $Q \leq 200,000$ 次询问, 每次需要回答 $S[l, r]$ 在 S 中共出现了多少次。

例题 3-快速定位 $s(S[l, r])$

题意

给出一个字符串 S , $|S| \leq 200,000$, 给出 $Q \leq 200,000$ 次询问, 每次需要回答 $S[l, r]$ 在 S 中共出现了多少次。

题解

本题可以用 SA 的做法, 转化为区间的查询, 略。

例题 3-快速定位 $s(S[l, r])$

题意

给出一个字符串 S , $|S| \leq 200,000$, 给出 $Q \leq 200,000$ 次询问, 每次需要回答 $S[l, r]$ 在 S 中共出现了多少次。

题解

本题可以用 SA 的做法, 转化为区间的查询, 略。

如果使用 SAM, 我们提前求出每个状态的 $|Right[s]|$, 询问就是要求我们快速定位 $S[l, r]$ 所在的状态。

例题 3-快速定位 $s(S[l, r])$

题意

给出一个字符串 S , $|S| \leq 200,000$, 给出 $Q \leq 200,000$ 次询问, 每次需要回答 $S[l, r]$ 在 S 中共出现了多少次。

题解

本题可以用 SA 的做法, 转化为区间的查询, 略。

如果使用 SAM, 我们提前求出每个状态的 $|Right[s]|$, 询问就是要求我们快速定位 $S[l, r]$ 所在的状态。

我们知道 $S[l, r]$ 一定是 S 的前缀 $S[1, r]$ 的后缀, 而 S 的前缀共有 n 个, 我们很容易找到 S 的每个前缀对应的 SAM 状态节点, 不妨设 $S[1, i]$ 对应于状态 $pos[i]$ 。

例题 3-快速定位 $s(S[l, r])$

题意

给出一个字符串 S , $|S| \leq 200,000$, 给出 $Q \leq 200,000$ 次询问, 每次需要回答 $S[l, r]$ 在 S 中共出现了多少次。

题解

本题可以用 SA 的做法, 转化为区间的查询, 略。

如果使用 SAM, 我们提前求出每个状态的 $|Right[s]|$, 询问就是要求我们快速定位 $S[l, r]$ 所在的状态。

我们知道 $S[l, r]$ 一定是 S 的前缀 $S[1, r]$ 的后缀, 而 S 的前缀共有 n 个, 我们很容易找到 S 的每个前缀对应的 SAM 状态节点, 不妨设 $S[1, i]$ 对应于状态 $pos[i]$ 。

由于 $S[l, r]$ 是 $S[1, r]$ 的后缀, 他对应的状态一定位于 $pos[i]$ 的后缀链接上, 也即我们要从 $pos[i]$ 到根 $root$ 这条树链上最浅的满足 $L[x] \geq r - l + 1$ 的状态。

例题 3-快速定位 $s(S[l, r])$

题意

给出一个字符串 S , $|S| \leq 200,000$, 给出 $Q \leq 200,000$ 次询问, 每次需要回答 $S[l, r]$ 在 S 中共出现了多少次。

题解

本题可以用 SA 的做法, 转化为区间的查询, 略。

如果使用 SAM, 我们提前求出每个状态的 $|Right[s]|$, 询问就是要求我们快速定位 $S[l, r]$ 所在的状态。

我们知道 $S[l, r]$ 一定是 S 的前缀 $S[1, r]$ 的后缀, 而 S 的前缀共有 n 个, 我们很容易找到 S 的每个前缀对应的 SAM 状态节点, 不妨设 $S[1, i]$ 对应于状态 $pos[i]$ 。

由于 $S[l, r]$ 是 $S[1, r]$ 的后缀, 他对应的状态一定位于 $pos[i]$ 的后缀链接上, 也即我们要从 $pos[i]$ 到根 $root$ 这条树链上最浅的满足

$L[x] \geq r - l + 1$ 的状态。

使用倍增即可。

例题 4-SAM+DP

题意

你有一台打字机，你需要用它打出一段只由小写字母构成的文本 S 。

设某个时刻，你已经打出来的文本为 T ，你可以执行两种操作：

1. 花费 $A[ch]$ 块钱，将 T 后面添加一个字符 ch ，得到 $T' = T + ch$ ，其中 $A['a' - 'z']$ 给定。
2. 花费 B 块钱，将 T 的一个子串 S 复制一份，添加到 T 的后面，得到 $T' = T + S$ 。

问你为了得到 S ，最少的花费是多少。

例题 4-SAM+DP

题意

你有一台打字机，你需要用它打出一段只由小写字母构成的文本 S 。

设某个时刻，你已经打出来的文本为 T ，你可以执行两种操作：

1. 花费 $A[ch]$ 块钱，将 T 后面添加一个字符 ch ，得到 $T' = T + ch$ ，其中 $A['a' - 'z']$ 给定。
2. 花费 B 块钱，将 T 的一个子串 S 复制一份，添加到 T 的后面，得到 $T' = T + S$ 。

问你为了得到 S ，最少的花费是多少。

题解

不难看出，本题可以用 DP 解决，设 $Cost[i]$ 表示得到 $S[1, i]$ 的最小代价。考虑如何计算 $Cost[i]$ ：

例题 4-SAM+DP

题意

你有一台打字机，你需要用它打出一段只由小写字母构成的文本 S 。

设某个时刻，你已经打出来的文本为 T ，你可以执行两种操作：

1. 花费 $A[ch]$ 块钱，将 T 后面添加一个字符 ch ，得到 $T' = T + ch$ ，其中 $A['a' - 'z']$ 给定。
2. 花费 B 块钱，将 T 的一个子串 S 复制一份，添加到 T 的后面，得到 $T' = T + S$ 。

问你为了得到 S ，最少的花费是多少。

题解

不难看出，本题可以用 DP 解决，设 $Cost[i]$ 表示得到 $S[1, i]$ 的最小代价。

考虑如何计算 $Cost[i]$ ：

1. 第一种操作带来的转移是： $Cost[i] \leftarrow Cost[i - 1] + A[S[i]]$ 。

例题 4-SAM+DP

题意

你有一台打字机，你需要用它打出一段只由小写字母构成的文本 S 。

设某个时刻，你已经打出来的文本为 T ，你可以执行两种操作：

1. 花费 $A[ch]$ 块钱，将 T 后面添加一个字符 ch ，得到 $T' = T + ch$ ，其中 $A[a' - 'z']$ 给定。
2. 花费 B 块钱，将 T 的一个子串 S 复制一份，添加到 T 的后面，得到 $T' = T + S$ 。

问你为了得到 S ，最少的花费是多少。

题解

不难看出，本题可以用 DP 解决，设 $Cost[i]$ 表示得到 $S[1, i]$ 的最小代价。

考虑如何计算 $Cost[i]$ ：

1. 第一种操作带来的转移是： $Cost[i] \leftarrow Cost[i - 1] + A[S[i]]$ 。
2. 第二种操作要求我们求最长的 $S[1, i]$ 的后缀 $S[i - x + 1, i]$ ，使得他在 $S[1, i - x]$ 中也出现过，带来的转移是：

$$Cost[i] \leftarrow \min(Cost[i - x \cdots i - 1]) + B。$$

例题 4-SAM+DP

题解

对于每个前缀 $S[1, i]$ ，如何求最长的后缀 $S[i - x + 1, i]$ ，使得他在 $S[1, i - x]$ 中也出现过？

例题 4-SAM+DP

题解

对于每个前缀 $S[1, i]$ ，如何求最长的后缀 $S[i - x + 1, i]$ ，使得他在 $S[1, i - x]$ 中也出现过？

用 *Right* 的概念对问题进行转化： $S[1, i - x]$ 中也出现过 $\rightarrow$

例题 4-SAM+DP

题解

对于每个前缀 $S[1, i]$ ，如何求最长的后缀 $S[i - x + 1, i]$ ，使得他在 $S[1, i - x]$ 中也出现过？

用 *Right* 的概念对问题进行转化： $S[1, i - x]$ 中也出现过 $\rightarrow$ *Right* 集合包含 $[1, i - x]$ 中的至少一个位置 $\rightarrow$

例题 4-SAM+DP

题解

对于每个前缀 $S[1, i]$ ，如何求最长的后缀 $S[i - x + 1, i]$ ，使得他在 $S[1, i - x]$ 中也出现过？

用 *Right* 的概念对问题进行转化： $S[1, i - x]$ 中也出现过 $\rightarrow$ *Right* 集合包含 $[1, i - x]$ 中的至少一个位置 $\rightarrow \text{Min}(\text{Right}(S[i - x + 1, i])) \leq i - x$ 。

例题 4-SAM+DP

题解

对于每个前缀 $S[1, i]$ ，如何求最长的后缀 $S[i - x + 1, i]$ ，使得他在 $S[1, i - x]$ 中也出现过？

用 *Right* 的概念对问题进行转化： $S[1, i - x]$ 中也出现过 $\rightarrow$ *Right* 集合包含 $[1, i - x]$ 中的至少一个位置 $\rightarrow \text{Min}(\text{Right}(S[i - x + 1, i])) \leq i - x$ 。
因此我们需要使用拓扑更新，得到 SAM 每个状态的 $\text{Min}(\text{Right}(s))$ ，然后再配合 ST 表，在 $S[1, i]$ 的后缀链接上倍增。

例题 5-区间询问版例题 3

题意

给出一个字符串 S , $|S| \leq 200,000$, 给出 $Q \leq 200,000$ 次询问, 每次需要回答 $S[l, r]$ 在 $S[L, R]$ 中共出现了多少次。

例题 5-区间询问版例题 3

题意

给出一个字符串 S , $|S| \leq 200,000$, 给出 $Q \leq 200,000$ 次询问, 每次需要回答 $S[l, r]$ 在 $S[L, R]$ 中共出现了多少次。

题解

我们依然将出现次数转化为满足条件的 $Right$ 的个数。

本题就是询问 $Right(s(S[l, r]))$ 中, 在区间 $[L + r - l, R]$ 范围内的有多少个值。

例题 5-区间询问版例题 3

题意

给出一个字符串 S , $|S| \leq 200,000$, 给出 $Q \leq 200,000$ 次询问, 每次需要回答 $S[l, r]$ 在 $S[L, R]$ 中共出现了多少次。

题解

我们依然将出现次数转化为满足条件的 $Right$ 的个数。

本题就是询问 $Right(s(S[l, r]))$ 中, 在区间 $[L + r - l, R]$ 范围内的有多少个值。

我们不能存储每个点的 $Right$ 集合, 这是 $O(n^2)$ 的。

前面讲过, 求每个点的 $Right$ 具体都有哪些值, 可以转化为子树询问, 进而变成 dfs 序上的区间询问。

例题 5-区间询问版例题 3

题意

给出一个字符串 S , $|S| \leq 200,000$, 给出 $Q \leq 200,000$ 次询问, 每次需要回答 $S[l, r]$ 在 $S[L, R]$ 中共出现了多少次。

题解

我们依然将出现次数转化为满足条件的 $Right$ 的个数。

本题就是询问 $Right(s(S[l, r]))$ 中, 在区间 $[L + r - l, R]$ 范围内的有多少个值。

我们不能存储每个点的 $Right$ 集合, 这是 $O(n^2)$ 的。

前面讲过, 求每个点的 $Right$ 具体都有哪些值, 可以转化为子树询问, 进而变成 dfs 序上的区间询问。

做法 1: 问题就是 dfs 序上对区间 $[dfs_l[s], dfs_r[s]]$ 询问, 有多少个数字值在 $[L + r - l, R]$ 范围内, 无脑持久化权值线段树, 甚至还可以处理强制在线。

例题 5-区间询问版例题 3

题意

给出一个字符串 S , $|S| \leq 200,000$, 给出 $Q \leq 200,000$ 次询问, 每次需要回答 $S[l, r]$ 在 $S[L, R]$ 中共出现了多少次。

题解

我们依然将出现次数转化为满足条件的 $Right$ 的个数。

本题就是询问 $Right(s(S[l, r]))$ 中, 在区间 $[L + r - l, R]$ 范围内的有多少个值。

我们不能存储每个点的 $Right$ 集合, 这是 $O(n^2)$ 的。

前面讲过, 求每个点的 $Right$ 具体都有哪些值, 可以转化为子树询问, 进而变成 dfs 序上的区间询问。

做法 1: 问题就是 dfs 序上对区间 $[dfs_l[s], dfs_r[s]]$ 询问, 有多少个数字值在 $[L + r - l, R]$ 范围内, 无脑持久化权值线段树, 甚至还可以处理强制在线。

做法 2: 如果不要强制在线, 可以先将每个询问挂在 $s(S[L, R])$ 上, 然后使用 *dsu on tree* 进行离线处理。

例题 6-单模式匹配

题意

给出两个长度均小于 100,000 的字符串 A 和 B ，求他们最长公共子串。

例题 6-单模式匹配

题意

给出两个长度均小于 100,000 的字符串 A 和 B ，求他们最长公共子串。

题解

本题依然可以使用 SA 求解，略。
本题还可以直接上广义 SAM，略。

例题 6-单模式匹配

题意

给出两个长度均小于 100,000 的字符串 A 和 B ，求他们最长公共子串。

题解

本题依然可以使用 SA 求解，略。

本题还可以直接上广义 SAM，略。

如果使用 SAM，思路是构造一个串的 SAM，将另一个串放在上面运行（匹配）。

例题 6-单模式匹配

题解

这里运行的意思是说，我们要对于每个 B 的前缀 $B[1, i]$ ，找到一个他的最大后缀 $B[i - x + 1, i]$ ，使得这个后缀是 A 的 SAM 可以表示的。

例题 6-单模式匹配

题解

这里运行的意思是说，我们要对于每个 B 的前缀 $B[1, i]$ ，找到一个他的最大后缀 $B[i - x + 1, i]$ ，使得这个后缀是 A 的 SAM 可以表示的。

设 $B[1, i - 1]$ 匹配到了 $state$ 状态，且这个状态能表示 $B[1, i - 1]$ 的最大长度为 len 的后缀 $B[i - 2 - len, i - 1]$ 。

接下来读入新字符 $B[i]$ ，我们找到合适的 $state'$ 以及 len' ，分两种情况：

例题 6-单模式匹配

题解

这里运行的意思是说，我们要对于每个 B 的前缀 $B[1, i]$ ，找到一个他的最大后缀 $B[i - x + 1, i]$ ，使得这个后缀是 A 的 SAM 可以表示的。

设 $B[1, i - 1]$ 匹配到了 $state$ 状态，且这个状态能表示 $B[1, i - 1]$ 的最大长度为 len 的后缀 $B[i - 2 - len, i - 1]$ 。

接下来读入新字符 $B[i]$ ，我们找到合适的 $state'$ 以及 len' ，分两种情况：

1. $trans(state, B[i]) \neq null$ ，也即 $state$ 拥有 $B[i]$ 的转移边，于是 $state' = trans(state, B[i])$ ， $len' = len + 1$ 。

例题 6-单模式匹配

题解

这里运行的意思是说，我们要对于每个 B 的前缀 $B[1, i]$ ，找到一个他的最大后缀 $B[i - x + 1, i]$ ，使得这个后缀是 A 的 SAM 可以表示的。

设 $B[1, i - 1]$ 匹配到了 $state$ 状态，且这个状态能表示 $B[1, i - 1]$ 的最大长度为 len 的后缀 $B[i - 2 - len, i - 1]$ 。

接下来读入新字符 $B[i]$ ，我们找到合适的 $state'$ 以及 len' ，分两种情况：

1. $trans(state, B[i]) \neq null$ ，也即 $state$ 拥有 $B[i]$ 的转移边，于是 $state' = trans(state, B[i])$ ， $len' = len + 1$ 。

2. $trans(state, B[i]) = null$ ，由于 $B[i]$ 是必须要处理的字符，因此我们只能让 len 变小，直到可以接受 $B[i]$ 为止。因此需要在 $state$ 的后缀链接上跳，

直到 $trans(state^*, B[i]) \neq null$

($state' = trans(state^*, B[i])$, $len' = L[state^*] + 1$)

或者跳到根都没有 $B[i]$ 的转移边 ($state' = root, len' = 0$)。

例题 7-多模式匹配

题意

给出 $N \leq 10$ 个长度均小于 100,000 的字符串，求他们的最长公共子串长度。

例题 7-多模式匹配

题意

给出 $N \leq 10$ 个长度均小于 100,000 的字符串，求他们的最长公共子串长度。

题解

非要用 SA 也不是不行，略。
也可以直接上广义 SAM，略。

例题 7-多模式匹配

题意

给出 $N \leq 10$ 个长度均小于 100,000 的字符串，求他们的最长公共子串长度。

题解

非要用 SA 也不是不行，略。

也可以直接上广义 SAM，略。

使用 SAM 的话，本题是一个多模式匹配，需要将上题的思路转变一下：构造其中一个串 A_1 的 SAM，然后将剩余的每个串 A_i 分别在上面运行。

例题 7-多模式匹配

题意

给出 $N \leq 10$ 个长度均小于 100,000 的字符串，求他们的最长公共子串长度。

题解

非要用 SA 也不是不行，略。

也可以直接上广义 SAM，略。

使用 SAM 的话，本题是一个多模式匹配，需要将上题的思路转变一下：构造其中一个串 A_1 的 SAM，然后将剩余的每个串 A_i 分别在上面运行。每次运行需要求对于 SAM 的每个节点，他能表示的最长的 A_i 的子串长度是多少。

放法是将 A_i 用前面相同的放法在 SAM 上运行，然后拓扑更新取 Max：如果 A_i 的某个前缀匹配到了 $state$ 状态的 len 长度，那么他一定也能匹配后缀链接上所有点的 $L[state^*]$ 。

例题 7-多模式匹配

题意

给出 $N \leq 10$ 个长度均小于 100,000 的字符串，求他们的最长公共子串长度。

题解

非要用 SA 也不是不行，略。

也可以直接上广义 SAM，略。

使用 SAM 的话，本题是一个多模式匹配，需要将上题的思路转变一下：构造其中一个串 A_1 的 SAM，然后将剩余的每个串 A_i 分别在上面运行。每次运行需要求对于 SAM 的每个节点，他能表示的最长的 A_i 的子串长度是多少。

放法是将 A_i 用前面相同的放法在 SAM 上运行，然后拓扑更新取 Max：如果 A_i 的某个前缀匹配到了 $state$ 状态的 len 长度，那么他一定也能匹配后缀链接上所有点的 $L[state^*]$ 。

最终每个状态将会得到 9 个匹配长度。将他们取 Min 就可以得到这个状态与 9 个串的最长公共子串长度，全局取最大即可。

例题 8-本质不同子串

题意

给出一个串 S ，求本质不同子串个数。

例题 8-本质不同子串

题意

给出一个串 S ，求本质不同子串个数。

构

造 S 的 SAM， $\sum_{state} L[state] - L[f(state)]$ 即为答案。

例题 9-本质不同子串的交

题意

给出两个串 S 和 T ，求公共本质不同子串个数。

例题 9-本质不同子串的交

题意

给出两个串 S 和 T ，求公共本质不同子串个数。

构

造 S 的 SAM，将 T 放在上面运行，对于每个状态 $state$ 求出能表示的最大 T 的子串长度 $len[state]$ ，放法同例题 7。

答案是 $\sum_{state} len[state] - L[fa(state)]$ 。

例题 10-本质不同子串的并

题意

给出两个串 S 和 T ，求他们本质不同子串的并集大小。

例题 10-本质不同子串的并

题意

给出两个串 S 和 T ，求他们本质不同子串的并集大小。

题解

利用容斥，答案 = S 的本质不同子串个数 + T 的本质不同子串个数 - 例题 9.

例题 11-新男人八题 A

题意

给出一个模板串 T ，和 n 个串 S_i ，保证输入的 S_i 都是 T 的子串。

$Alice$ 和 Bob 进行一场游戏，两人轮流操作，每次操作必须：

1. 选择某一个串 S_i 。
 2. 将 S_i 后面添加一个字符 ch ，要求 $S'_i = S_i + ch$ 必须也是 T 的子串。
- 不能操作者输， $Alice$ 先行，问谁会赢得这个游戏。

例题 11-新男人八题 A

题意

给出一个模板串 T ，和 n 个串 S_i ，保证输入的 S_i 都是 T 的子串。

$Alice$ 和 Bob 进行一场游戏，两人轮流操作，每次操作必须：

1. 选择某一个串 S_i 。
 2. 将 S_i 后面添加一个字符 ch ，要求 $S'_i = S_i + ch$ 必须也是 T 的子串。
- 不能操作者输， $Alice$ 先行，问谁会赢得这个游戏。

题解

不难看出，这是一个组合 Nim 游戏，每个串单独来看，就是一个字符串版本的取石子游戏。

例题 11-新男人八题 A

题意

给出一个模板串 T ，和 n 个串 S_i ，保证输入的 S_i 都是 T 的子串。

$Alice$ 和 Bob 进行一场游戏，两人轮流操作，每次操作必须：

1. 选择某一个串 S_i 。
 2. 将 S_i 后面添加一个字符 ch ，要求 $S'_i = S_i + ch$ 必须也是 T 的子串。
- 不能操作者输， $Alice$ 先行，问谁会赢得这个游戏。

题解

不难看出，这是一个组合 Nim 游戏，每个串单独来看，就是一个字符串版本的取石子游戏。

因此要求我们去求每个独立游戏的 SG 函数，每次操作是在字符串后面新增一个字母，很容易联想到 SAM。

因此本题对 T 构造 SAM，然后再 $trans$ 转移图 (DAG) 上进行 DP (dfs) 求解每个状态的 SG 函数值。

判断所有独立游戏的 SG 值异或和是否为 0 即可判定游戏胜负。

Problem List

疑难杂症

1. Codeforces gym 100962 D Deep Purple: 子串最大 Border 查询
题解 → $O(n \log n^2)$ 离线 SAM + 树剖线段树, 或者在线 $O(n \log n)$ 基本字典子串 (dbf)。
2. 洛谷 P5115
题解 → SAM + 边分治 + 虚树 DP (好像边分树会简单一点。)
3. 牛客 Contest203F
题解 → SAM + Manacher / SA。
4. WF 2019 G First of Her Name
题解 → 广义 SAM (Trie 上 SAM)。
5. 区间本质不同子串
CF-Blog, 板子 不需要 LCT。